SID4: 1066

PORT_BASE: 8166

PREFIX: s1066

SEED: 1066

VERIFY_SEED: 261066

DOMAIN_ID: 2

Hardware: Macintosh HD, Apple M4, 24 GB, Tahoe 26.6.2

Local Model Used: Llama3.2:latest

Tagged Commit Hash: https://github.com/paknathan/data260-1066

Part 1)

Code:

```
(base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - Part 1 code"
Nathan Pak - Part 1 code
(base) Nathans-MBP:code nathanpak$ python -c "import inspect, models, routers.documents as d; print(inspect.getsource(models.TransitIncidentDoc)); print(inspect.getsource(d.create_
incident)); print(inspect.getsource(d.update_incident))"
class TransitIncidentDoc(BaseModel):
    """Request body for create (POST) and full update (PUT)."""
    model_config = ConfigDict(use_enum_values=True, str_strip_whitespace=True, extra="forbid")

    title: str = Field(..., min_length=1, max_length=100, description="Required short summary")
    description: Optional[str] = Field(None, max_length=2000, description="Optional details")
    status: IncidentStatus = Field(IncidentStatus.reported, description="reported | investigating | resolved")
    category: TransitCategory = Field(..., description="bus | light_rail | subway")
    event_time: datetime = Field(..., description="ISO 8601 date/time the incident occurred")

@router.post("", status_code=status.HTTP_201_CREATED)
def create_incident(item: TransitIncidentDoc):
    now = utcnow()
    data = item.model_dump()
    data["created_at"] = now
    data["updated_at"] = now
    try:
        result = doc_collection.insert_one(data)
        return serialize(doc_collection.find_one({"_id": result.inserted_id}))
    except PyMongoError as exc:
        raise _db_error(exc)

@router.put("/{item_id}")
def update_incident(item_id: str, item: TransitIncidentDoc):
    oid = _object_id(item_id)
    data = item.model_dump()
    data["updated_at"] = utcnow()
    try:
        doc = doc_collection.find_one_and_update(
            {"_id": oid}, {"$set": data}, return_document=ReturnDocument.AFTER
        )
    except PyMongoError as exc:
        raise _db_error(exc)
    if not doc:
        raise HTTPException(status_code=404, detail=f"Incident {item_id} not found")
    return serialize(doc)

(base) Nathans-MBP:code nathanpak$
```

```
(base) Nathans-MBP:code nathanpak$ curl -i http://127.0.0.1:8166/api/items
HTTP/1.1 200 OK
date: Mon, 19 Oct 2026 21:35:04 GMT
server: uvicorn
content-length: 2
content-type: application/json

](base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - HW6 Part 1"
Nathan Pak - HW6 Part 1
(base) Nathans-MBP:code nathanpak$ curl -i -X POST http://127.0.0.1:8166/api/items -H 'Content-Type:
 application/json' -d '{"title":"Bus 23 breakdown","description":"Engine failure on Winchester Blvd"
,"category":"bus","event_time":"2026-10-19T08:15:00"}'
HTTP/1.1 201 Created
date: Mon, 19 Oct 2026 21:36:21 GMT
server: uvicorn
content-length: 267
content-type: application/json

{"_id":"6ad68d551f67c983840b715c","title":"Bus 23 breakdown","description":"Engine failure on Winche
ster Blvd","status":"reported","category":"bus","event_time":"2026-10-19T08:15:00","created_at":"202
6-10-19T21:36:21.677000","updated_at":"2026-10-19T21:36:21.677000"}(base) Nathans-MBP:code nathanpak
$ curl -i -X POST http://127.0.0-H 'Content-Type: application/json' -d '{"title":"Light rail signal
fault","category":"light_rail","status":"investigating","event_time":"2026-10-19T09:00:00"}'
HTTP/1.1 201 Created
date: Mon, 19 Oct 2026 21:36:21 GMT
server: uvicorn
content-length: 255
content-type: application/json

{"_id":"6ad68d551f67c983840b715d","title":"Light rail signal fault","description":null,"status":"inv
estigating","category":"light_rail","event_time":"2026-10-19T09:00:00","created_at":"2026-10-19T21:3
6:21.777000","updated_at":"2026-10-19T21:36:21.777000"}(base) Nathans-MBP:code nathanpak$ curl -i ht
tp://127.0.0.1:8166/api/items
HTTP/1.1 200 OK
date: Mon, 19 Oct 2026 21:36:27 GMT
server: uvicorn
content-length: 525
content-type: application/json

[{"_id":"6ad68d551f67c983840b715d","title":"Light rail signal fault","description":null,"status":"in
vestigating","category":"light_rail","event_time":"2026-10-19T09:00:00","created_at":"2026-10-19T21:
36:21.777000","updated_at":"2026-10-19T21:36:21.777000"},{"_id":"6ad68d551f67c983840b715c","title":"
Bus 23 breakdown","description":"Engine failure on Winchester Blvd","status":"reported","category":"
bus","event_time":"2026-10-19T08:15:00","created_at":"2(base(base(base(base) Nat(base(base(base(base
(base) Nathans-MBP:code nathanpak$
```

```
(base) Nathans-MBP:code nathanpak$ curl -i http://127.0.0.1:8166/api/items/$ID
HTTP/1.1 404 Not Found
date: Mon, 19 Oct 2026 21:39:04 GMT
server: uvicorn
content-length: 56
content-type: application/json

{"detail":"Incident 6ad68d551f67c983840b715d not found"}(base) Nathans-MBP:code nathanpak$ curl -i -
pe: application/json' -d '{"title":"Bus 23 breakdown","status":"resolved","category":"bus","event_ti
me":"2026-10-19T08:15:00"}'
HTTP/1.1 404 Not Found
date: Mon, 19 Oct 2026 21:39:04 GMT
server: uvicorn
content-length: 56
content-type: application/json

{"detail":"Incident 6ad68d551f67c983840b715d not found"}(base) Nathans-MBP:code nathanpak$ curl -i -
X DELETE http://127.0.0.1:8166/api/items/$ID
HTTP/1.1 404 Not Found
date: Mon, 19 Oct 2026 21:39:04 GMT
server: uvicorn
content-length: 56
content-type: application/json

{"detail":"Incident 6ad68d551f67c983840b715d not found"}(base) Nathans-MBP:code nathanpak$ curl -i -
 application/json' -d '{"title":"x","category":"ferry"}'
HTTP/1.1 422 Unprocessable Entity
date: Mon, 19 Oct 2026 21:39:04 GMT
server: uvicorn
content-length: 203
content-type: application/json

{"detail":"Validation failed","errors":[{"field":"category","message":"Input should be 'bus', 'light
_rail' or 'subway'","type":"enum"},{"field":"event_time","message":"Field required","type":"missing"
}]}(base) Nathans-MBP:code nathanpak$ curl -i http://127.0.0.1:8166/api/items/123
HTTP/1.1 400 Bad Request
date: Mon, 19 Oct 2026 21:39:04 GMT
server: uvicorn
content-length: 67
content-type: application/json

{"detail":"Invalid ID format: '123' is not a 24-char hex ObjectId"}(base) Nathans-MBP:code nathanpak
cidents.find().toArray()'sh s1066_doc --quiet --eval 'db.transit_in
[
  {
    _id: ObjectId('6ad68d551f67c983840b715c'),
    title: 'Bus 23 breakdown',
    description: 'Engine failure on Winchester Blvd',
    status: 'reported',
    category: 'bus',
    event_time: ISODate('2026-10-19T08:15:00.000Z'),
    created_at: ISODate('2026-10-19T21:36:21.677Z'),
    updated_at: ISODate('2026-10-19T21:36:21.677Z')
  }
]
(base) Nathans-MBP:code nathanpak$
```

Part 2)

Code:

```
(base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - Part 2 code"
Nathan Pak - Part 2 code
(base) Nathans-MBP:code nathanpak$ python -c "import inspect, routers.memory as r; print(inspect.getsource(r.chat)); print(inspect.getsource(r.aggregate))"
python -c "import inspect, memory_core as m; print(inspect.getsource(m.chat_turn))"
python -c "import inspect, memory_core as m; [print(inspect.getsource(f)) for f in (m.extract_episodes, m.maybe_summarize)]"
python -c "import inspect, memory_core as m; [print(inspect.getsource(f)) for f in (m.retrieve_episodes, m.compose_prompt)]"
@router.post("/chat")
def chat(req: ChatRequest):
    try:
        if req.message.strip().startswith("/"):
            recent = memory_core.load_short_term(req.session_id, 20) if req.session_id else []
            mode = memory_core.cfg()["extract_mode"]
            result = file_memory.handle_command(
                req.message, recent, extractor=lambda t: memory_core.extract_episodes(t, mode))
            return {"session_id": req.session_id, **result}
        return memory_core.chat_turn(req.user_id, req.session_id, req.message)
    except LLMError as exc:
        raise HTTPException(status_code=503, detail=f"Local model unavailable: {exc}")
    except PyMongoError as exc:
        raise HTTPException(status_code=503, detail=f"MongoDB error: {exc}")

@router.get("/aggregate/{user_id}")
def aggregate(user_id: str):
    pipeline = [
        {"$match": {"user_id": user_id}},
        {"$group": {
            "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at",
                                      "timezone": "America/Los_Angeles"}},
            "total": {"$sum": 1},
            "user_messages": {"$sum": {"$cond": [{"$eq": ["$role", "user"]}, 1, 0]}},
            "assistant_messages": {"$sum": {"$cond": [{"$eq": ["$role", "assistant"]}, 1, 0]}},
        }},
        {"$sort": {"_id": 1}},
        {"$project": {"_id": 0, "date": "$_id", "total": 1, "user_messages": 1, "assistant_messages": 1}},
    ]
    try:
        daily = list(messages_col.aggregate(pipeline))
        recent = list(summaries_col.find({"user_id": user_id}).sort("created_at", DESCENDING).limit(5))
    except PyMongoError as exc:
        raise HTTPException(status_code=503, detail=f"MongoDB error: {exc}")
    return {"user_id": user_id, "daily_message_counts": daily,
            "recent_summaries": [_clean(s) for s in recent]}

(base) Nathans-MBP:code nathanpak$ python -c "import inspect, memory_core as m; print(inspect.getsource(m.chat_turn))"
def chat_turn(user_id: str, session_id: Optional[str], message: str, overrides: Optional[dict] = None,
              run_label: str = "api") -> Dict:
    c = cfg(overrides)
    session_id = session_id or f"sess-{uuid.uuid4().hex[:10]}"
    t0 = time.perf_counter()

    # 1. save user message
    msg_id = save_message(user_id, session_id, "user", message)
    # 2. short-term window (+ optional compression policy)
    window = apply_policy(load_short_term(session_id, c["short_term_n"], exclude_id=msg_id), c["policy"])
    # 3. long-term summaries
    sess = latest_summary(user_id, "session", session_id)
    life = latest_summary(user_id, "user")
    # 4. episodic retrieval
    min_imp = 0.4 if c["policy"] == "compress" else 0.0
    episodes = retrieve_episodes(user_id, llm.embed(message), c["top_k"], min_imp)
    # 5. compose prompt (system primer may include AGENT.md + MEMORY.md)
    primer = file_memory.build_system_prompt(SYSTEM_PRIMER) if c["file_memory"] else SYSTEM_PRIMER
    prompt, breakdown = compose_prompt(primer, sess["text"] if sess else None,
                                       life["text"] if life else None, window, message, episodes)
    # 6. call model
    out = llm.chat(prompt)
```

```
    primer = file_memory.build_system_prompt(SYSTEM_PRIMER) if c["file_memory"] else SYSTEM_PRIMER
    prompt, breakdown = compose_prompt(primer, sess["text"] if sess else None,
                                       life["text"] if life else None, window, message, episodes)
    # 6. call model
    out = llm.chat(prompt)
    # 7. save reply
    save_message(user_id, session_id, "assistant", out["text"])
    # 8. episodic extraction for this user message
    new_facts = store_episodes(user_id, session_id, extract_episodes(message, c["extract_mode"]))
    # 9. summarization trigger
    flags = maybe_summarize(user_id, session_id, c)
    total_ms = round((time.perf_counter() - t0) * 1000, 1)

    usage = {
        "prompt_tokens": out["prompt_tokens"], "completion_tokens": out["completion_tokens"],
        **breakdown, "model_latency_ms": out["latency_ms"], "request_latency_ms": total_ms,
        "model": out["model"], "embed_backend": llm.embed_backend(),
    }
    log_usage({"ts": utcnow().isoformat(), "run": run_label, "user_id": user_id, "session_id": session_id,
               "short_term_n": c["short_term_n"], "policy": c["policy"], **usage}, c["log_path"])

    return {
        "reply": out["text"],
        "session_id": session_id,
        "memory_used": {
            "short_term_count": len(window),
            "short_term": window,
            "session_summary": sess["text"] if sess else None,
            "lifetime_summary": life["text"] if life else None,
            "episodic_facts": episodes,
            "new_episodes_stored": new_facts,
            **flags,
        },
        "usage": usage,
    }

(base) Nathans-MBP:code nathanpak$ python -c "import inspect, memory_core as m; [print(inspect.getsource(f)) for f in (m.extract_episodes, m.maybe_summarize)]"
def extract_episodes(message: str, mode: str = "llm") -> List[Dict]:
    """Extract up to 3 short durable facts (with importance 0..1) from one user message."""
    if mode != "llm":
        return _heuristic_facts(message)
    prompt = (
        "Extract at most 3 short, durable facts about the user from the message below "
        "(identity, preferences, routes, incidents they report, plans). Ignore questions and small talk. "
        'Return JSON: {"facts": [{"fact": "<= 20 words, third person", "importance": 0.0-1.0}]}. '
        'Return {"facts": []} if there are none.\n\nMessage: ' + message
    )
    try:
        out = llm.chat([{"role": "user", "content": prompt}], temperature=0.0, max_tokens=200, json_mode=True)
        data = json.loads(out["text"])
        facts = []
        for f in data.get("facts", [])[:3]:
            fact = str(f.get("fact", "")).strip()
            if fact:
                imp = float(f.get("importance", 0.5))
                facts.append({"fact": fact[:200], "importance": max(0.0, min(1.0, imp))})
        return facts
    except (llm.LLMError, ValueError, AttributeError, TypeError):
        return _heuristic_facts(message)

def maybe_summarize(user_id: str, session_id: str, c: dict) -> Dict[str, bool]:
    """Every M user messages: write a session summary; every K session summaries: refresh lifetime summary."""
    flags = {"session_summary_created": False, "lifetime_summary_created": False}
    n_user = messages_col.count_documents({"session_id": session_id, "role": "user"})
    if n_user == 0 or n_user % c["summarize_every"] != 0:
        return flags
```

```
    recent = load_short_term(session_id, 2 * c["summarize_every"])
    prev = latest_summary(user_id, "session", session_id)
    convo = "\n".join(f"{m['role']}: {m['content']}" for m in recent)
    text = (f"Previous session summary:\n{prev['text']}\n\n" if prev else "") + f"Recent conversation:\n{convo}"
    summary = _summarize(text, "Update the session summary with the recent conversation.")
    summaries_col.insert_one({"user_id": user_id, "session_id": session_id, "scope": "session",
                              "text": summary, "created_at": utcnow()})
    flags["session_summary_created"] = True

    n_sess = summaries_col.count_documents({"user_id": user_id, "scope": "session"})
    if n_sess % c["lifetime_every"] == 0:
        sess = list(summaries_col.find({"user_id": user_id, "scope": "session"})
                    .sort("created_at", DESCENDING).limit(5))
        life = latest_summary(user_id, "user")
        body = (f"Current lifetime summary:\n{life['text']}\n\n" if life else "") + \
               "Session summaries:\n" + "\n\n".join(s["text"] for s in reversed(sess))
        lifetime = _summarize(body, "Condense these into one lifetime profile of the user (durable facts only).")
        summaries_col.insert_one({"user_id": user_id, "session_id": None, "scope": "user",
                                  "text": lifetime, "created_at": utcnow()})
        flags["lifetime_summary_created"] = True
    return flags

(base) Nathans-MBP:code nathanpak$ python -c "import inspect, memory_core as m; [print(inspect.getsource(f)) for f in (m.retrieve_episodes, m.compose_prompt)]"
def retrieve_episodes(user_id: str, query_vec: List[float], k: int, min_importance: float = 0.0) -> List[Dict]:
    """Top-k episodes for this user by cosine similarity to the current message embedding."""
    cands = episodes_col.find({"user_id": user_id, "importance": {"$gte": min_importance}},
                              {"fact": 1, "importance": 1, "embedding": 1}).sort("created_at", DESCENDING).limit(500)
    scored = [(llm.cosine(query_vec, e["embedding"]), e) for e in cands]
    scored.sort(key=lambda t: (t[0], t[1]["importance"]), reverse=True)
    seen, out = set(), []
    for score, e in scored:
        key = e["fact"].lower()
        if key in seen:
            continue
        seen.add(key)
        out.append({"fact": e["fact"], "importance": e["importance"], "score": round(score, 4)})
        if len(out) >= k:
            break
    return out

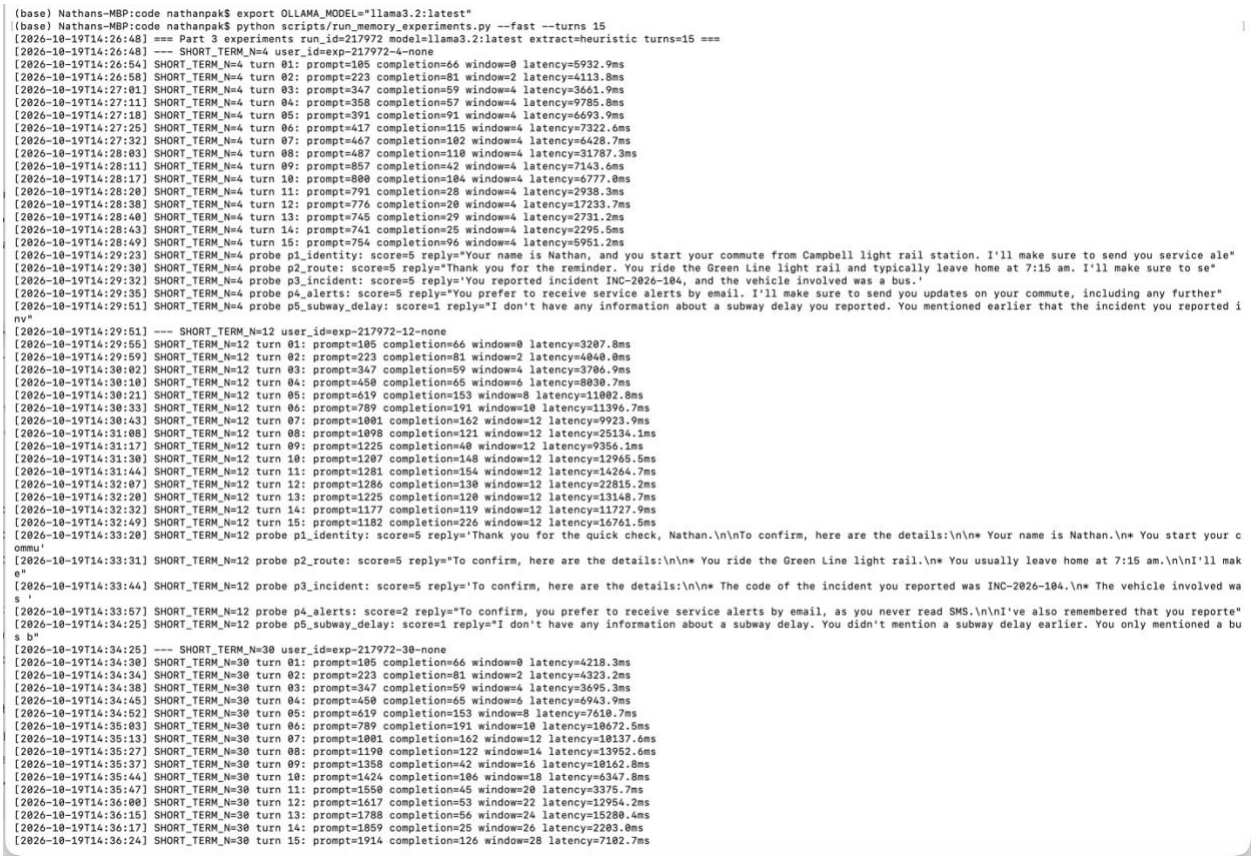
def compose_prompt(system_primer: str, session_summary: Optional[str], lifetime_summary: Optional[str],
                   window: List[Dict[str, str]], message: str, episodes: List[Dict]) -> Tuple[List[Dict], Dict]:
    """system primer + long-term summaries + short-term window + current message + episodic facts."""
    long_term = ""
    if lifetime_summary:
        long_term += f"## Lifetime summary of the user\n{lifetime_summary}\n"
    if session_summary:
        long_term += f"## Summary of this session so far\n{session_summary}\n"
    episodic = ""
    if episodes:
        episodic = "## Relevant facts remembered about the user\n" + "\n".join(f"- {e['fact']}" for e in episodes)

    system = system_primer + ("\n\n" + long_term if long_term else "")
    current = message + ("\n\n" + episodic if episodic else "")
    messages = [{"role": "system", "content": system}] + window + [{"role": "user", "content": current}]
    breakdown = {
        "system_primer_tokens": llm.approx_tokens(system_primer),
        "long_term_tokens": llm.approx_tokens(long_term),
        "short_term_tokens": sum(llm.approx_tokens(m["content"]) for m in window),
        "episodic_tokens": llm.approx_tokens(episodic),
        "current_message_tokens": llm.approx_tokens(message),
    }
    return messages, breakdown

(base) Nathans-MBP:code nathanpak$
```

```
(base) Nathans-MBP:code nathanpak$ docker exec mongo mongosh s1066_doc --quiet --eval 'db.messages.d
istinct("user_id")'
[
  'exp-217972-12-compress',
  'exp-217972-12-none',
  'exp-217972-30-none',
  'exp-217972-4-none'
]
(base) Nathans-MBP:code nathanpak$ U=paste_one_exp_user_id_here
(base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - HW6 Part 2"
Nathan Pak - HW6 Part 2
(base) Nathans-MBP:code nathanpak$ curl -s http://127.0.0.1:8166/api/memory/$U | python -m json.tool
 | head -80
{
    "user_id": "paste_one_exp_user_id_here",
    "recent_messages": [],
    "latest_session_summary": null,
    "latest_lifetime_summary": null,
    "episodic_facts": []
}
(base) Nathans-MBP:code nathanpak$ curl -s http://127.0.0.1:8166/api/aggregate/$U | python -m json.t
ool
{
    "user_id": "paste_one_exp_user_id_here",
    "daily_message_counts": [],
    "recent_summaries": []
}
(base) Nathans-MBP:code nathanpak$ docker exec mongo mongosh s1066_doc --quiet --eval 'db.getCollect
ionNames(); db.messages.find({},{content:1,role:1,user_id:1}).limit(4).toArray(); db.summaries.find(
).limit(2).toArray(); db.episodes.find({},{embedding:0}).limit(3).toArray()'
[
  {
    _id: ObjectId('6ad68b1efb4407ef33e16ce9'),
    user_id: 'exp-217972-4-none',
    session_id: 'exp-217972-4-none-sess',
    fact: 'My name is Nathan and I commute every weekday from Campbell light rail station.',
    importance: 0.8,
    created_at: ISODate('2026-10-19T21:26:54.128Z')
  },
  {
    _id: ObjectId('6ad68b22fb4407ef33e16cec'),
    user_id: 'exp-217972-4-none',
    session_id: 'exp-217972-4-none-sess',
    fact: 'I ride the Green Line light rail and I usually leave home at 7:15 am.',
    importance: 0.5,
    created_at: ISODate('2026-10-19T21:26:58.252Z')
  },
  {
    _id: ObjectId('6ad68b25fb4407ef33e16cef'),
    user_id: 'exp-217972-4-none',
    session_id: 'exp-217972-4-none-sess',
    fact: 'Please always send me service alerts by email, I never read SMS.',
    importance: 0.8,
    created_at: ISODate('2026-10-19T21:27:01.915Z')
  }
]
(base) Nathans-MBP:code nathanpak$
```

Conversation screenshots:



```
(base) Nathans-MBP:code nathanpak$ export OLLAMA_MODEL="llama3.2:latest"
(base) Nathans-MBP:code nathanpak$ python scripts/run_memory_experiments.py --fast --turns 15
[2026-10-19T14:26:48] === Part 3 experiments run_id=217972 model=llama3.2:latest extract=heuristic turns=15 ===
[2026-10-19T14:26:48] --- SHORT_TERM_N=4 user_id=exp-217972-4-none
[2026-10-19T14:26:54] SHORT_TERM_N=4 turn 01: prompt=105 completion=66 window=0 latency=5932.9ms
[2026-10-19T14:26:58] SHORT_TERM_N=4 turn 02: prompt=223 completion=81 window=2 latency=4113.8ms
[2026-10-19T14:27:01] SHORT_TERM_N=4 turn 03: prompt=347 completion=59 window=4 latency=3661.9ms
[2026-10-19T14:27:11] SHORT_TERM_N=4 turn 04: prompt=358 completion=57 window=4 latency=9785.8ms
[2026-10-19T14:27:18] SHORT_TERM_N=4 turn 05: prompt=391 completion=91 window=4 latency=6693.9ms
[2026-10-19T14:27:25] SHORT_TERM_N=4 turn 06: prompt=417 completion=115 window=4 latency=7322.6ms
[2026-10-19T14:27:32] SHORT_TERM_N=4 turn 07: prompt=467 completion=102 window=4 latency=6428.7ms
[2026-10-19T14:28:03] SHORT_TERM_N=4 turn 08: prompt=487 completion=110 window=4 latency=31787.3ms
[2026-10-19T14:28:11] SHORT_TERM_N=4 turn 09: prompt=857 completion=42 window=4 latency=7143.6ms
[2026-10-19T14:28:17] SHORT_TERM_N=4 turn 10: prompt=800 completion=104 window=4 latency=6777.0ms
[2026-10-19T14:28:20] SHORT_TERM_N=4 turn 11: prompt=791 completion=28 window=4 latency=2938.3ms
[2026-10-19T14:28:38] SHORT_TERM_N=4 turn 12: prompt=776 completion=20 window=4 latency=17233.7ms
[2026-10-19T14:28:40] SHORT_TERM_N=4 turn 13: prompt=745 completion=29 window=4 latency=2731.2ms
[2026-10-19T14:28:43] SHORT_TERM_N=4 turn 14: prompt=741 completion=25 window=4 latency=2295.5ms
[2026-10-19T14:28:49] SHORT_TERM_N=4 turn 15: prompt=754 completion=96 window=4 latency=5951.2ms
[2026-10-19T14:29:23] SHORT_TERM_N=4 probe p1_identity: score=5 reply="Your name is Nathan, and you start your commute from Campbell light rail station. I'll make sure to send you service ale"
[2026-10-19T14:29:30] SHORT_TERM_N=4 probe p2_route: score=5 reply="Thank you for the reminder. You ride the Green Line light rail and typically leave home at 7:15 am. I'll make sure to se"
[2026-10-19T14:29:32] SHORT_TERM_N=4 probe p3_incident: score=5 reply='You reported incident INC-2026-104, and the vehicle involved was a bus.'
[2026-10-19T14:29:35] SHORT_TERM_N=4 probe p4_alerts: score=5 reply="You prefer to receive service alerts by email. I'll make sure to send you updates on your commute, including any further"
[2026-10-19T14:29:51] SHORT_TERM_N=4 probe p5_subway_delay: score=1 reply="I don't have any information about a subway delay you reported. You mentioned earlier that the incident you reported i
nv"
[2026-10-19T14:29:51] --- SHORT_TERM_N=12 user_id=exp-217972-12-none
[2026-10-19T14:29:55] SHORT_TERM_N=12 turn 01: prompt=105 completion=66 window=0 latency=3207.8ms
[2026-10-19T14:29:59] SHORT_TERM_N=12 turn 02: prompt=223 completion=81 window=2 latency=4040.0ms
[2026-10-19T14:30:02] SHORT_TERM_N=12 turn 03: prompt=347 completion=59 window=4 latency=3706.9ms
[2026-10-19T14:30:10] SHORT_TERM_N=12 turn 04: prompt=450 completion=65 window=6 latency=8030.7ms
[2026-10-19T14:30:21] SHORT_TERM_N=12 turn 05: prompt=619 completion=153 window=8 latency=11002.8ms
[2026-10-19T14:30:33] SHORT_TERM_N=12 turn 06: prompt=789 completion=191 window=10 latency=11396.7ms
[2026-10-19T14:30:43] SHORT_TERM_N=12 turn 07: prompt=1001 completion=162 window=12 latency=9923.9ms
[2026-10-19T14:31:08] SHORT_TERM_N=12 turn 08: prompt=1098 completion=121 window=12 latency=25134.1ms
[2026-10-19T14:31:17] SHORT_TERM_N=12 turn 09: prompt=1225 completion=40 window=12 latency=9356.1ms
[2026-10-19T14:31:30] SHORT_TERM_N=12 turn 10: prompt=1207 completion=148 window=12 latency=12965.5ms
[2026-10-19T14:31:44] SHORT_TERM_N=12 turn 11: prompt=1281 completion=154 window=12 latency=14264.7ms
[2026-10-19T14:32:07] SHORT_TERM_N=12 turn 12: prompt=1286 completion=130 window=12 latency=22815.2ms
[2026-10-19T14:32:20] SHORT_TERM_N=12 turn 13: prompt=1225 completion=120 window=12 latency=13148.7ms
[2026-10-19T14:32:32] SHORT_TERM_N=12 turn 14: prompt=1177 completion=119 window=12 latency=11727.9ms
[2026-10-19T14:32:49] SHORT_TERM_N=12 turn 15: prompt=1182 completion=226 window=12 latency=16761.5ms
[2026-10-19T14:33:20] SHORT_TERM_N=12 probe p1_identity: score=5 reply='Thank you for the quick check, Nathan.\n\nTo confirm, here are the details:\n\n* Your name is Nathan.\n* You start your c
ommu'
[2026-10-19T14:33:31] SHORT_TERM_N=12 probe p2_route: score=5 reply="To confirm, here are the details:\n\n* You ride the Green Line light rail.\n* You usually leave home at 7:15 am.\n\nI'll mak
e"
[2026-10-19T14:33:44] SHORT_TERM_N=12 probe p3_incident: score=5 reply='To confirm, here are the details:\n\n* The code of the incident you reported was INC-2026-104.\n* The vehicle involved wa
s '
[2026-10-19T14:33:57] SHORT_TERM_N=12 probe p4_alerts: score=2 reply="To confirm, you prefer to receive service alerts by email, as you never read SMS.\n\nI've also remembered that you reporte"
[2026-10-19T14:34:25] SHORT_TERM_N=12 probe p5_subway_delay: score=1 reply="I don't have any information about a subway delay. You didn't mention a subway delay earlier. You only mentioned a bu
s b"
[2026-10-19T14:34:25] --- SHORT_TERM_N=30 user_id=exp-217972-30-none
[2026-10-19T14:34:30] SHORT_TERM_N=30 turn 01: prompt=105 completion=66 window=0 latency=4218.3ms
[2026-10-19T14:34:34] SHORT_TERM_N=30 turn 02: prompt=223 completion=81 window=2 latency=4323.2ms
[2026-10-19T14:34:38] SHORT_TERM_N=30 turn 03: prompt=347 completion=59 window=4 latency=3695.3ms
[2026-10-19T14:34:45] SHORT_TERM_N=30 turn 04: prompt=450 completion=65 window=6 latency=6943.9ms
[2026-10-19T14:34:52] SHORT_TERM_N=30 turn 05: prompt=619 completion=153 window=8 latency=7610.7ms
[2026-10-19T14:35:03] SHORT_TERM_N=30 turn 06: prompt=789 completion=191 window=10 latency=10672.5ms
[2026-10-19T14:35:13] SHORT_TERM_N=30 turn 07: prompt=1001 completion=162 window=12 latency=10137.6ms
[2026-10-19T14:35:27] SHORT_TERM_N=30 turn 08: prompt=1190 completion=122 window=14 latency=13952.6ms
[2026-10-19T14:35:37] SHORT_TERM_N=30 turn 09: prompt=1358 completion=42 window=16 latency=10162.8ms
[2026-10-19T14:35:44] SHORT_TERM_N=30 turn 10: prompt=1424 completion=106 window=18 latency=6347.8ms
[2026-10-19T14:35:47] SHORT_TERM_N=30 turn 11: prompt=1550 completion=45 window=20 latency=3375.7ms
[2026-10-19T14:36:00] SHORT_TERM_N=30 turn 12: prompt=1617 completion=53 window=22 latency=12954.2ms
[2026-10-19T14:36:15] SHORT_TERM_N=30 turn 13: prompt=1788 completion=56 window=24 latency=15280.4ms
[2026-10-19T14:36:17] SHORT_TERM_N=30 turn 14: prompt=1859 completion=25 window=26 latency=2203.0ms
[2026-10-19T14:36:24] SHORT_TERM_N=30 turn 15: prompt=1914 completion=126 window=28 latency=7102.7ms
```

```
[2026-10-19T14:31:30] SHORT_TERM_N=12 turn 10: prompt=1207 completion=148 window=12 latency=12965.5ms
[2026-10-19T14:31:44] SHORT_TERM_N=12 turn 11: prompt=1281 completion=154 window=12 latency=14264.7ms
[2026-10-19T14:32:07] SHORT_TERM_N=12 turn 12: prompt=1286 completion=130 window=12 latency=22815.2ms
[2026-10-19T14:32:20] SHORT_TERM_N=12 turn 13: prompt=1225 completion=120 window=12 latency=13148.7ms
[2026-10-19T14:32:32] SHORT_TERM_N=12 turn 14: prompt=1177 completion=119 window=12 latency=11727.9ms
[2026-10-19T14:32:49] SHORT_TERM_N=12 turn 15: prompt=1182 completion=226 window=12 latency=16761.5ms
[2026-10-19T14:33:20] SHORT_TERM_N=12 probe p1_identity: score=5 reply='Thank you for the quick check, Nathan.\n\nTo confirm, here are the details:\n\n* Your name is Nathan.\n* You start your c
ommu'
[2026-10-19T14:33:31] SHORT_TERM_N=12 probe p2_route: score=5 reply="To confirm, here are the details:\n\n* You ride the Green Line light rail.\n* You usually leave home at 7:15 am.\n\nI'll mak
e"
[2026-10-19T14:33:44] SHORT_TERM_N=12 probe p3_incident: score=5 reply='To confirm, here are the details:\n\n* The code of the incident you reported was INC-2026-104.\n* The vehicle involved wa
s '
[2026-10-19T14:33:57] SHORT_TERM_N=12 probe p4_alerts: score=2 reply="To confirm, you prefer to receive service alerts by email, as you never read SMS.\n\nI've also remembered that you reporte"
[2026-10-19T14:34:25] SHORT_TERM_N=12 probe p5_subway_delay: score=1 reply="I don't have any information about a subway delay. You didn't mention a subway delay earlier. You only mentioned a bu
s b"
[2026-10-19T14:34:25] --- SHORT_TERM_N=30 user_id=exp-217972-30-none
[2026-10-19T14:34:30] SHORT_TERM_N=30 turn 01: prompt=105 completion=66 window=0 latency=4218.3ms
[2026-10-19T14:34:34] SHORT_TERM_N=30 turn 02: prompt=223 completion=81 window=2 latency=4323.2ms
[2026-10-19T14:34:38] SHORT_TERM_N=30 turn 03: prompt=347 completion=59 window=4 latency=3695.3ms
[2026-10-19T14:34:45] SHORT_TERM_N=30 turn 04: prompt=450 completion=65 window=6 latency=6943.9ms
[2026-10-19T14:34:52] SHORT_TERM_N=30 turn 05: prompt=619 completion=153 window=8 latency=7610.7ms
[2026-10-19T14:35:03] SHORT_TERM_N=30 turn 06: prompt=789 completion=191 window=10 latency=10672.5ms
[2026-10-19T14:35:13] SHORT_TERM_N=30 turn 07: prompt=1001 completion=162 window=12 latency=10137.6ms
[2026-10-19T14:35:27] SHORT_TERM_N=30 turn 08: prompt=1190 completion=122 window=14 latency=13952.6ms
[2026-10-19T14:35:37] SHORT_TERM_N=30 turn 09: prompt=1358 completion=42 window=16 latency=10162.8ms
[2026-10-19T14:35:44] SHORT_TERM_N=30 turn 10: prompt=1424 completion=106 window=18 latency=6347.8ms
[2026-10-19T14:35:47] SHORT_TERM_N=30 turn 11: prompt=1550 completion=45 window=20 latency=3375.7ms
[2026-10-19T14:36:00] SHORT_TERM_N=30 turn 12: prompt=1617 completion=53 window=22 latency=12954.2ms
[2026-10-19T14:36:15] SHORT_TERM_N=30 turn 13: prompt=1788 completion=56 window=24 latency=15280.4ms
[2026-10-19T14:36:17] SHORT_TERM_N=30 turn 14: prompt=1859 completion=25 window=26 latency=2203.0ms
[2026-10-19T14:36:24] SHORT_TERM_N=30 turn 15: prompt=1914 completion=126 window=28 latency=7102.7ms
[2026-10-19T14:36:51] SHORT_TERM_N=30 probe p1_identity: score=5 reply="Quick check!\n\nYou're right, I've got all the relevant facts about you, Nathan:\n\n* Your name is Nathan.\n* You commute
 eve"
[2026-10-19T14:37:08] SHORT_TERM_N=30 probe p2_route: score=5 reply="To confirm:\n\n* You ride the Green Line light rail.\n* You usually leave home at 7:15 am.\n\nI'll make sure to keep these d
e"
[2026-10-19T14:37:26] SHORT_TERM_N=30 probe p3_incident: score=5 reply="I've got all the relevant facts!\n\n* The code of the incident you reported is: INC-2026-104\n* The vehicle involved in t
he"
[2026-10-19T14:37:45] SHORT_TERM_N=30 probe p4_alerts: score=5 reply="Thank you for reminding me of your preferences!\n\nYou prefer to receive service alerts by email, and I'll make sure to se"
[2026-10-19T14:38:15] SHORT_TERM_N=30 probe p5_subway_delay: score=5 reply='Thank you for reminding me of the relevant facts!\n\n* The subway delay you mentioned was 25 minutes long.\n* The sta
tion w'
[2026-10-19T14:38:15] --- Compress policy (N=12) user_id=exp-217972-12-compress
[2026-10-19T14:38:19] Compress policy (N=12) turn 01: prompt=105 completion=66 window=0 latency=3614.7ms
[2026-10-19T14:38:23] Compress policy (N=12) turn 02: prompt=223 completion=81 window=2 latency=4062.9ms
[2026-10-19T14:38:27] Compress policy (N=12) turn 03: prompt=347 completion=59 window=4 latency=3979.5ms
[2026-10-19T14:38:35] Compress policy (N=12) turn 04: prompt=405 completion=66 window=6 latency=8556.3ms
[2026-10-19T14:38:44] Compress policy (N=12) turn 05: prompt=516 completion=102 window=8 latency=8706.3ms
[2026-10-19T14:38:55] Compress policy (N=12) turn 06: prompt=597 completion=177 window=10 latency=10608.6ms
[2026-10-19T14:39:05] Compress policy (N=12) turn 07: prompt=754 completion=175 window=12 latency=10587.6ms
[2026-10-19T14:39:27] Compress policy (N=12) turn 08: prompt=822 completion=113 window=12 latency=21387.2ms
[2026-10-19T14:39:33] Compress policy (N=12) turn 09: prompt=821 completion=42 window=12 latency=6768.9ms
[2026-10-19T14:39:41] Compress policy (N=12) turn 10: prompt=688 completion=104 window=12 latency=7911.6ms
[2026-10-19T14:39:45] Compress policy (N=12) turn 11: prompt=666 completion=18 window=12 latency=3376.4ms
[2026-10-19T14:39:57] Compress policy (N=12) turn 12: prompt=651 completion=40 window=12 latency=12887.8ms
[2026-10-19T14:40:01] Compress policy (N=12) turn 13: prompt=642 completion=31 window=12 latency=3898.5ms
[2026-10-19T14:40:05] Compress policy (N=12) turn 14: prompt=647 completion=26 window=12 latency=3516.4ms
[2026-10-19T14:40:09] Compress policy (N=12) turn 15: prompt=635 completion=41 window=12 latency=3932.3ms
[2026-10-19T14:40:30] Compress policy (N=12) probe p1_identity: score=5 reply="You are Nathan, and you start your commute from Campbell light rail station on the Green Line. I'll make sure to s
end yo"
[2026-10-19T14:40:36] Compress policy (N=12) probe p2_route: score=5 reply="You ride the Green Line light rail, and you usually leave home at 7:15 am. I'll make sure to send you service alerts
by "
[2026-10-19T14:40:40] Compress policy (N=12) probe p3_incident: score=5 reply='The code of the incident you reported was INC-2026-104, and the vehicle involved was a bus.'
[2026-10-19T14:40:43] Compress policy (N=12) probe p4_alerts: score=5 reply="I'll send you service alerts by email, as you prefer."
[2026-10-19T14:40:57] Compress policy (N=12) probe p5_subway_delay: score=1 reply='There is no mention of a subway delay. You only mentioned a bus incident (INC-2026-104) and a light rail incid
ent (which'
[2026-10-19T14:40:57] Wrote /Users/nathanpak/Desktop/Data260/data260-1066/code/reports/hw06/METRICS.md; best=SHORT_TERM_N=30 worst=SHORT_TERM_N=30
(base) Nathans-MBP:code nathanpak$
```

Part 3)

Code:

```
(base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - Part 3 code"
Nathan Pak - Part 3 code
(base) Nathans-MBP:code nathanpak$ python -c "import inspect, memory_core as m; print(inspect.getsource(m.apply_policy))"
python -c "import inspect, sys; sys.path.insert(0,'scripts'); import run_memory_experiments as e; print(inspect.getsource(e.score_probe))"
def apply_policy(window: List[Dict[str, str]], policy: str) -> List[Dict[str, str]]:
    """Memory-saving policy 'compress': keep the newest 4 messages verbatim and
    compress every older message in the window to a short gist (first ~90 chars)."""
    if policy != "compress" or len(window) <= COMPRESS_KEEP_VERBATIM:
        return window
    older, recent = window[:-COMPRESS_KEEP_VERBATIM], window[-COMPRESS_KEEP_VERBATIM:]
    gisted = []
    for m in older:
        text = m["content"].replace("\n", " ")
        if len(text) > COMPRESS_GIST_CHARS:
            text = text[:COMPRESS_GIST_CHARS].rsplit(" ", 1)[0] + " …"
        gisted.append({"role": m["role"], "content": text})
    return gisted + recent

(base) Nathans-MBP:code nathanpak$ python -c "import inspect, sys; sys.path.insert(0,'scripts'); import run_memory_experiments as e; print(inspect.getsource(e.score_probe))"
def score_probe(reply: str, probe: dict) -> int:
    text = reply.lower()
    matched = sum(1 for crit in probe["criteria"] if any(k.lower() in text for k in crit))
    score = 1 + round(4 * matched / len(probe["criteria"]))
    if any(f.lower() in text for f in probe.get("forbidden", [])):
        score = min(score, 2)
    return score

(base) Nathans-MBP:code nathanpak$
```

```
[2026-10-19T14:40:57] Wrote /Users/nathanpak/Desktop/Data260/data260-1066/code/reports/hw06/METRICS.md; best=SHORT_TERM_N=30 worst=SHORT_TERM_N=30
(base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - HW6 Part 3"
Nathan Pak - HW6 Part 3
(base) Nathans-MBP:code nathanpak$ cat reports/hw06/METRICS.md
# HW6 METRICS (generated by scripts/run_memory_experiments.py)

Model: `llama3.2:latest` · episode extraction: heuristic · turns per run: 15 · run_id 217972

## Part 3.3/3.4 - memory settings

| Setting | Mean prompt tokens/turn | Mean latency (ms) | Probe quality (1-5) |
|---|---|---|---|
| SHORT_TERM_N=4 | 550.6 | 8053.2 | 4.2 |
| SHORT_TERM_N=12 | 881 | 11832.2 | 3.6 |
| SHORT_TERM_N=30 | 1082.3 | 7932.0 | 5 |
| Compress policy (N=12) | 567.9 | 7586.3 | 4.2 |

### Prompt-token breakdown (mean per turn, approx.)

| Setting | Short-term | Long-term summary | Episodic | Completion tokens |
|---|---|---|---|---|
| SHORT_TERM_N=4 | 173.3 | 228.5 | 60.1 | 68.3 |
| SHORT_TERM_N=12 | 667.9 | 108.8 | 60.1 | 122.3 |
| SHORT_TERM_N=30 | 890.3 | 95.4 | 60.1 | 90.1 |
| Compress policy (N=12) | 313.9 | 105.7 | 60.1 | 76.1 |

## Part 3.4 - compression policy vs baseline

- Baseline: SHORT_TERM_N=12; policy: Compress policy (N=12)
- Prompt tokens saved per turn: 313.1 (35.5%)
- Probe quality change: +0.6

## Part 3.5 - cost of 10,000 conversations ($1/M input, $5/M output)

| Setting | Cost / conversation | Cost / 10,000 conversations |
|---|---|---|
| SHORT_TERM_N=4 | $0.0134 | $133.81 |
| SHORT_TERM_N=12 | $0.0224 | $223.87 |
| SHORT_TERM_N=30 | $0.0230 | $229.92 |
| Compress policy (N=12) | $0.0142 | $142.26 |

- **Best setting:** SHORT_TERM_N=30 - cheapest setting whose probe quality is within 0.5 of the best score (5).
- **Worst setting:** SHORT_TERM_N=30 - highest cost per conversation.
- Best vs worst for 10,000 conversations: $229.92 vs $229.92.
(base) Nathans-MBP:code nathanpak$
```

Cat_memory_probes.yaml

```
(base) Nathans-MBP:code nathanpak$ cat memory_probes.yaml
# Part 3.2 - fixed memory probes (created BEFORE running the experiments).
# Each probe is asked after the 30-message conversation in the same session.
# Auto-score = 1 + round(4 * matched_criteria / total_criteria)   (1..5)
# A criterion matches if ANY of its accepted keywords appears in the reply (case-insensitive).
# "forbidden" keywords (a wrong/contradicting answer) cap the score at 2.

rubric:
  5: All expected facts recalled correctly, no wrong facts.
  4: Most expected facts recalled (e.g. 2 of 2 with minor wording issues, or 3 of 4).
  3: About half of the expected facts recalled.
  2: Only a small part recalled, or the answer contains a contradicting/wrong fact.
  1: Nothing relevant recalled, or the model says it does not know.

probes:
  - id: p1_identity
    question: "Quick check: what is my name and which station do I start my commute from?"
    criteria:
      - ["nathan"]
      - ["campbell"]
    forbidden: ["winchester"]
  - id: p2_route
    question: "Which light rail line do I ride, and what time do I usually leave home?"
    criteria:
      - ["green line", "green"]
      - ["7:15", "7.15", "seven fifteen"]
    forbidden: ["blue line"]
  - id: p3_incident
    question: "What was the code of the incident I reported, and what kind of vehicle was involved?"
    criteria:
      - ["inc-2026-104", "2026-104"]
      - ["bus"]
    forbidden: []
  - id: p4_alerts
    question: "How do I prefer to receive service alerts?"
    criteria:
      - ["email", "e-mail"]
    forbidden: ["sms", "text message"]
  - id: p5_subway_delay
    question: "How many minutes was the subway delay I told you about, and which station was it at?"
    criteria:
      - ["25", "twenty-five", "twenty five"]
      - ["market street", "market st"]
    forbidden: []
(base) Nathans-MBP:code nathanpak$
```

Compression policy: newest 4 window messages kept verbatim, older window messages compressed to ~90-character gists, and only episodes with importance ≥ 0.4 retrieved.

Best/worst selection: worst = highest cost per conversation; best = cheapest setting whose probe score is within 0.5 of the top score. Runs used 15-message conversations (instead of 30) due to time; cost is still estimated for 30-turn conversations.

Part 4)

```
(base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - Part 4 code"
Nathan Pak - Part 4 code
(base) Nathans-MBP:code nathanpak$ python -c "import inspect, file_memory as f; [print(inspect.getsource(x)) for x in (f.load_memory_files, f.build_system_prompt, f.read_last_conso
lidated, f.needs_consolidation)]"
def load_memory_files() -> Tuple[str, str]:
    """Return (AGENT.md text, MEMORY.md text); missing files load as empty strings."""
    return _read(AGENT_PATH), _read(MEMORY_PATH)

def build_system_prompt(base_primer: str) -> str:
    """Append AGENT.md (instructions) and MEMORY.md (durable facts) to the system primer."""
    agent, memory = load_memory_files()
    parts = [base_primer]
    if agent.strip():
        parts.append("# Agent instructions (AGENT.md)\n" + _strip_comments(agent).strip())
    if memory.strip():
        parts.append("# Durable memory (MEMORY.md)\n" + _strip_comments(memory).strip())
    return "\n\n".join(parts)

def read_last_consolidated(text: str) -> Optional[datetime]:
    m = TS_RE.search(text)
    if not m:
        return None
    try:
        ts = datetime.fromisoformat(m.group(1).replace("Z", "+00:00"))
        return ts if ts.tzinfo else ts.replace(tzinfo=timezone.utc)
    except ValueError:
        return None

def needs_consolidation(text: str, now: Optional[datetime] = None) -> Tuple[bool, List[str]]:
    now = now or datetime.now(timezone.utc)
    reasons = []
    n = count_content_lines(text)
    if n > MAX_LINES:
        reasons.append(f"{n} content lines > {MAX_LINES}")
    ts = read_last_consolidated(text)
    if ts is None:
        reasons.append("no last_consolidated timestamp recorded")
    elif now - ts > MAX_AGE:
        reasons.append(f"last consolidated {ts.isoformat()} is more than 24h old")
    return bool(reasons), reasons

(base) Nathans-MBP:code nathanpak$ python -c "import inspect, file_memory as f; print(inspect.getsource(f.handle_command))"
def handle_command(command: str, recent_messages: Optional[List[Dict[str, str]]] = None,
                   extractor=None) -> Dict:
    cmd = command.strip().split()[0].lower()
    if cmd == "/help":
        return {"command": "/help", "reply": HELP_TEXT}

    if cmd == "/memory":
        agent, memory = load_memory_files()
        due, reasons = needs_consolidation(memory)
        return {"command": "/memory",
                "reply": f"--- AGENT.md ---\n{agent}\n--- MEMORY.md ---\n{memory}",
                "memory_content_lines": count_content_lines(memory),
                "last_consolidated": str(read_last_consolidated(memory)),
                "consolidation_due": due, "reasons": reasons}

    if cmd == "/compact":
        report = consolidate_file(MEMORY_PATH, ROOT / "reports" / "hw06" / "raw" / "compact_report.json")
        reply = (f"/compact: MEMORY.md {report['lines_before']} lines -> {report['lines_after']} lines "
                 f"({report['content_lines_before']} -> {report['content_lines_after']} non-comment lines; "
                 f"{report['facts_before']} -> {report['facts_after']} facts). "
                 f"Merged {len(report['duplicates_merged'])} duplicated facts, resolved "
                 f"{len(report['contradictions_resolved'])} contradictions, flagged for review: "
                 f"{report['flagged_possibly_incorrect'] or 'none'}.")
        return {"command": "/compact", "reply": reply, "report": report}
```

```
    if cmd == "/dream":
        msgs = [m for m in (recent_messages or []) if m["role"] == "user"]
        facts: List[str] = []
        for m in msgs[-10:]:
            found = extractor(m["content"]) if extractor else []
            facts += [f["fact"] for f in found if f.get("importance", 0) >= 0.5]
        added = append_facts(facts)
        return {"command": "/dream", "reply": f"/dream: scanned {len(msgs[-10:])} recent user messages, "
                                              f"added {len(added)} new facts to MEMORY.md.", "added": added}

    return {"command": cmd, "reply": f"Unknown command {cmd}.\n\n{HELP_TEXT}"}

(base) Nathans-MBP:code nathanpak$ cat AGENT.md
# AGENT.md — Municipal Transit Incident Manager (s1066)

<!-- Durable project + agent instructions. Loaded into the system prompt by file_memory.build_system_prompt(). -->

## Project description
1. This project is the Municipal Transit Incident Manager for DATA-260 (SID4 1066). It tracks bus, light-rail and subway incidents, congestion indexes, and a chat assistant with me
mory.
2. The app always runs on PORT_BASE 8166; the MongoDB database is `s1066_doc`; SEED is 1066 and VERIFY_SEED is 261066.

## Technology stack
3. Backend is FastAPI + Uvicorn; relational data uses SQLAlchemy (MySQL), document and memory data use PyMongo (MongoDB).
4. The chat model is a local Ollama model called only through `llm_adapter.py` — never call a hosted/paid API.

## Coding conventions
5. Use snake_case names, type hints on every parameter and return value, and a one-line docstring on every public function.
6. Validate inputs explicitly and raise `ValueError` (or `HTTPException` in routes) with a clear message; never fail silently.
7. Library code must not use `print()`; return values instead so code is testable.

## Testing approach
8. Every new helper gets pytest tests in `tests/` covering a normal case, a boundary case and an invalid input. Run with `pytest -q`.
9. `make verify-hw06` (scripts/verify_hw06.py) must pass on the tagged commit before submission.

## Prohibited actions
10. Never commit secrets (`.env`, passwords, `key.pem`), never hard-code credentials, and never drop or overwrite MongoDB collections from application code.
11. Do not modify the Part 1 `transit_incidents` collection schema when working on memory features.
(base) Nathans-MBP:code nathanpak$ python -c "import inspect, transit_utils as t; print(inspect.getsource(t.classify_delay))"
def classify_delay(delay_minutes: int) -> str:
    """Return the delay band (on_time/minor/moderate/severe) for a delay in whole minutes."""
    if isinstance(delay_minutes, bool) or not isinstance(delay_minutes, int):
        raise ValueError(f"delay_minutes must be an int, got {type(delay_minutes).__name__}")
    if delay_minutes < 0:
        raise ValueError(f"delay_minutes must be >= 0, got {delay_minutes}")
    band = DELAY_BANDS[0][1]
    for threshold, name in DELAY_BANDS:
        if delay_minutes >= threshold:
            band = name
    return band

(base) Nathans-MBP:code nathanpak$


(base) Nathans-MBP:code nathanpak$ echo "Nathan Pak - HW6 Part 4"
Nathan Pak - HW6 Part 4
(base) Nathans-MBP:code nathanpak$ python scripts/make_memory_fixture.py
MEMORY_original.md written: 55 lines
(base) Nathans-MBP:code nathanpak$ python scripts/memory_cli.py /compact
/compact: MEMORY.md 55 lines -> 35 lines (54 -> 33 non-comment lines; 53 -> 26 facts). Merged 17 duplicated facts, resolved 4 contradictions, flagged for review: ['auth.session_ttl
'].
{
  "lines_before": 55,
  "lines_after": 35,
  "facts_before": 53,
  "facts_after": 26,
  "flagged_possibly_incorrect": [
    "auth.session_ttl"
  ]
}
(base) Nathans-MBP:code nathanpak$ wc -l MEMORY_original.md MEMORY.md
      55 MEMORY_original.md
      35 MEMORY.md
      90 total
(base) Nathans-MBP:code nathanpak$ pytest -q tests
..................                                                       [100%]
18 passed in 0.08s
```